

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
 Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	Assessment:	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	Total Marks:	100
CO2 Statement:	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)		
Student Name:	K. A. Analiya	Reg. No.:	192521395
Section / Batch:	B. Tech - IT	Date:	26/07/2026

Instructions to Students

- Answer ALL questions. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems.
- Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required.
- Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis.
- Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Questions	Marks
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis Q1 Q2,Q3	40 60 (2X30)
GRAND TOTAL:		100 Marks

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I K. A. Analiya (192521395) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: K. A. Analiya

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Question No.	Understanding of Problem Context (20%)	Application of Concepts (30%)	Problem-Solving Approach (20%)	Accuracy of Solution (20%)	Clarity (10%)	Total Marks
Q1 (40 Marks)						
Q2 (30 Marks)						
Q3 (30 Marks)						
Overall Total (100)						

Faculty In-charge	Course Coordinator	Head of Department
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

CO2 $\Rightarrow$ ATI $\Rightarrow$ Analytical

Based questions

Name : K. A. Analiya

Reg no : 192521395

code : CSA-0608

course : design and
analysis of
algorithm.

90 Searching Techniques.

Given sorted dataset [4, 8, 12, 16, 20, 24, 28, 32, 36] compare linear search and Binary search to find element 28.

Problem interpretation:

The dataset is already sorted. Both, linear search and binary search can be applied.

Linear search:

Array: [4, 8, 12, 16, 20, 24, 28, 32, 36].

Target = 28.

Comparison	Element	Result
1	4	Not Found
2	8	Not Found
3	12	Not Found
4	16	Not Found
5	20	Not Found
6	24	Not Found
7	28	Found

comparisons = 7.

Binary search:

Initial: Low=0, High=8

Step 1:

Middle = $(0+8)/2 = 4$.

Element = 20

$28 > 20$

search Right Half

Step 2:

Low=5 High=8

Middle = 6.

Element = 28

Target Found.

comparisons = 2.

Time complexity:

Algorithm	Best	Average	worst
Linear search	$O(1)$	$O(n)$	$O(n)$
Binary search	$O(1)$	$O(\log n)$	$O(\log n)$

Comparison:

- Linear search checks elements one by one.
- Binary search repeatedly divides the search space into two halves.
- Since the data is sorted, Binary search performs significantly fewer comparisons.

Result verification:

Target Element = 28

Linear search comparisons = 7.

Binary search comparisons = 2.

Both algorithms return the correct result.

Conclusion:

Binary search is the preferred algorithm for sorted datasets because it requires fewer comparisons and has logarithmic time complexity, making it suitable for large datasets.

Q5 closest pair:

Emergency service locations are

$\{(1,3), (4,5), (7,8), (2,4), (9,1), (5,6)\}$

Find closest pair using Brute Force.

problem interpretation:

The objective is to calculate the distance between every possible pair of points and determine the minimum distance.

Formula:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distance calculations:

$$A(1,3) - B(4,5) = \sqrt{13} = 3.61$$

$$A - C = \sqrt{61} = 7.81$$

$$A - B = \sqrt{2} = 1.41$$

$$A - E = \sqrt{68} = 8.25$$

$$A - F = \sqrt{25} = 5$$

$$B - C = \sqrt{18} = 4.24$$

$$B - D = \sqrt{5} = 2.24$$

$$B - E = \sqrt{41} = 6.40$$

$$B - F = \sqrt{2} = 1.41$$

$$C - D = \sqrt{41} = 6.40$$

$$C - E = \sqrt{53} = 7.28$$

$$C - F = \sqrt{8} = 2.83$$

$$D - E = \sqrt{58} = 7.62$$

$$D - F = \sqrt{13} = 3.61$$

$$E - F = \sqrt{41} = 6.40$$

closest pairs :

Minimum distance = 1.41.

closest pairs.

$\Rightarrow (1,3)$ and $(2,4)$.

$\rightarrow (4,5)$ and $(5,6)$

Number of comparisons :

For 6 points,

$$\frac{6(6-1)}{2} = 15.$$

Total comparisons = 15.

Time complexity :

$\Rightarrow$ Best case = $O(n^2)$

$\Rightarrow$ Average case = $O(n^2)$

$\Rightarrow$ worst case = $O(n^2)$

Verification :

The smallest distance is 1.41, so the nearest emergency service locations are correctly identified.

Conclusion :

Brute Force compares every possible pair and guarantees the correct answer. It is suitable for small datasets but becomes inefficient for large numbers of locations.

123. Selection sort :

Unordered values with Repeated Minimum

Problem interpretation.

The given dataset is $[22, 5, 31, 14, 5, 27]$. The objective is to sort the element in ascending order using the selection sort algorithm. Selection sort repeatedly selects the smallest element from the unsorted portion and places it at the correct position.

Justification for using selection sort :

⇒ selection sort is a simple comparison-based sorting algorithm.

⇒ It finds the smallest element in each pass and places it in the correct position.

⇒ It works well for small datasets.

Step by step sorting :

Initial Array

$[22, 5, 31, 14, 5, 27]$

pass 1 :

smallest element = 5 (index 1)

swap 22 and 5

$[5, 22, 31, 14, 5, 27]$

pass 2 :

unsorted part :

$[22, 31, 14, 5, 27]$

smallest element = 5
(index 4)

swap 22 and 5

$[5, 5, 31, 14, 22, 27]$

pass 3 :

unsorted part :

[31, 14, 22, 27]

smallest element = 14

swap 31 and 14.

[5, 5, 14, 31, 22, 27].

Final sorted Array:

[5, 5, 14, 22, 27, 31].

comparisons :

For $n=6$.

$$\begin{aligned}\text{comparisons} &= (6-1) + (6-2) + (6-3) + (6-4) + (6-5) \\ &= 5 + 4 + 3 + 2 + 1 = 15 \text{ comparisons.}\end{aligned}$$

swaps :

pass 1 $\rightarrow$ 1 swap.

pass 2 $\rightarrow$ 1 swap.

pass 3 $\rightarrow$ 1 swap.

pass 4 $\rightarrow$ 1 swap.

pass 5 $\rightarrow$ 1 swap.

Total swap = 5.

time complexity

case

time complexity

Best case

$O(n^2)$

Average case

$O(n^2)$

Worst case

$O(n^2)$

Verification:

Original Array :

[22, 5, 31, 14, 5, 27].

sorted Array :

[5, 5, 14, 22, 27, 31]

The array is correctly arranged in ascending order.
Both occurrences of the minimum value (5) are preserved.

pass 4 :

unsorted part :

[31, 22, 27].

smallest element = 22

swap 31 and 22

[5, 5, 14, 22, 31, 27]

pass 5 :

unsorted part :

[31, 27]

smallest element = 27

swap 31 and 27

[5, 5, 14, 22, 27, 31]